

Loan Approval and Risk Assessment System

A Complete Beginner's Guide to the Machine Learning Project

This document explains, from absolute zero, what this project is, how machine learning works, exactly what our code does step by step, why we chose the Random Forest algorithm, whether we built a new model or used an existing one, and what our results mean. Read this once before your presentation and you'll be able to answer almost any question about it.

1. What Is Machine Learning? (The Absolute Basics)

Normally, when we write a computer program, **we** tell the computer the exact rules. For example: *"If income is above 50,000, approve the loan."* We write every rule by hand.

Machine Learning (ML) flips this around. Instead of us writing the rules, we give the computer a lot of **examples** (past loan applications, along with whether they were approved or rejected), and the computer **figures out the rules itself** by finding patterns in the data.

Traditional programming: Rules + Data → Answers

Machine Learning: Data + Answers → Rules (the computer learns the rules)

In our project, the "rules" the computer learns are things like: *"applicants with good credit history and high income tend to get approved."* We never typed that rule anywhere — the model discovered it by studying 491 real past loan applications.

1.1 Types of Machine Learning

- **Supervised Learning** — the computer learns from labeled examples, i.e. we already know the correct answer for each example (this is what we used: every past loan application already says Approved or Rejected).
- **Unsupervised Learning** — the computer finds patterns in data with no labeled answers (e.g. grouping customers into segments).
- **Reinforcement Learning** — the computer learns by trial and error, getting rewards or penalties (e.g. how game-playing AIs learn).

Our project uses **Supervised Learning**, and more specifically a **Classification** problem — we are predicting one of two categories (Approved or Rejected), not a number.

1.2 Key Vocabulary You'll Hear

Term	Plain-English Meaning
Feature	An input the model uses to make a decision (e.g. Income, Credit History)
Label / Target	The answer we want to predict (here: Loan_Status — Approved or Rejected)
Training	The process where the model studies examples and learns patterns
Model	The "thing" that results from training — it can now make predictions on new data
Training Set	The portion of data used to teach the model (we used 80%)
Test Set	Data the model has NEVER seen, used to check if it actually learned (we used 20%)
Overfitting	When a model memorizes the training data instead of learning general patterns, so it performs badly on
Prediction	The model's output/guess for a new, unseen example

2. Is This an Existing Model, or Did We Build a New One?

This is a very common point of confusion, and an important one to get right for your presentation. The answer has two parts:

2.1 Random Forest the ALGORITHM already exists

"Random Forest" is a well-known, published machine learning **algorithm** (a general method/technique), invented in 2001 by Leo Breiman. It is not something we invented — it's a standard tool available in every major ML library (we used Python's **scikit-learn** library, which already has Random Forest built in).

2.2 OUR MODEL (the trained result) is brand new — we built it

An **algorithm** is like a recipe. A **model** is the actual dish you cooked using that recipe with your own specific ingredients. We used the Random Forest recipe/algorithm, but we:

- Chose our own dataset (491 real loan applications)
- Cleaned and prepared that data ourselves
- Trained a brand-new model from scratch on this data (nobody had trained this exact model before — it exists only because we ran the training process)
- Tuned settings ourselves (200 trees, max depth of 6, etc.)
- Tested and evaluated it ourselves to see how good it is

Simple answer for your evaluator: "We used an existing, well-known algorithm (Random Forest), but we trained a completely new model ourselves, from scratch, on our own dataset. We did not download or reuse anyone else's already-trained model."

3. What Is a Random Forest? (How It Actually Works)

3.1 Start with a Decision Tree

Before understanding a "forest", you need to understand a single "tree". A **Decision Tree** is basically a flowchart of yes/no questions that ends in a decision:

```
Is Credit History good?
→ Yes → Is Income above $4,000?
    → Yes → APPROVE
    → No → Is Loan Amount small? → ...
→ No → REJECT
```

The computer doesn't write these questions by hand — it automatically finds the best questions to ask, and in what order, by testing which questions best separate approved applicants from rejected ones in the training data.

Problem with a single tree: it can become overly specific to the exact data it was trained on (overfitting), so it might not work well on new, unseen applicants.

3.2 Now Make a Forest

A Random Forest fixes this by building **many** decision trees (we used **200 trees**) instead of just one, where:

- Each tree only sees a random subset of the training data (not all of it)
- Each tree only considers a random subset of features at each question
- This means every tree ends up slightly different from the others

When a new applicant comes in, **all 200 trees make their own prediction**, and the forest takes a **majority vote**. If 140 trees say "Approve" and 60 say "Reject", the final answer is Approve.

Why this works better: Individual trees might make mistakes or be overly opinionated about quirks in the data they happened to see. But when you average 200 different opinions, the mistakes tend to cancel out, and the strong, real patterns rise to the top. This is sometimes called "the wisdom of the crowd."

3.3 Settings We Used, in Plain English

Setting	Value	What it means
n_estimators	200	Build 200 different decision trees
max_depth	6	Each tree can only ask up to 6 questions deep (keeps trees simple, avoids overfitting)
min_samples_leaf	4	Each final decision must be based on at least 4 similar applicants (avoids trusting a single v
class_weight	balanced	Our data has more approvals than rejections; this tells the model to not get lazy and just pr

4. Our Dataset — Where It Came From & What's In It

We used a well-known, publicly available loan applicant dataset (the same type of dataset used in countless academic and industry loan-approval projects, so it's a credible, standard choice). It contains **491 real loan applications** with 13 columns of information.

4.1 The Columns (Features)

Column	Meaning
Gender	Applicant's gender
Married	Marital status
Dependents	Number of people financially dependent on the applicant
Education	Graduate or Not Graduate
Self_Employed	Whether the applicant is self-employed
ApplicantIncome	Applicant's monthly income
CoapplicantIncome	Co-applicant's (if any) monthly income
LoanAmount	Requested loan amount
Loan_Amount_Term	Repayment period, in days
Credit_History	Whether the applicant has a good repayment history (1) or not (0)
Property_Area	Rural, Semiurban, or Urban
Loan_Status (target)	Whether the loan was Approved (1) or Rejected (0) — this is what we predict

Notice that **Loan_Status** is special — it's the answer key. Every other column is a "feature" (an input clue) the model uses to guess Loan_Status for applicants it hasn't seen before.

5. Step-by-Step: What Our Code (loan_model.py) Actually Does

Step 1 — Load the Data

We read the CSV file into a table (using a library called pandas) and drop the Loan_ID column, since an ID number is just a label with no predictive meaning — keeping it would confuse the model.

Step 2 — Clean the Data

Real-world data is messy. Some applicants had missing values (e.g. some didn't state their Credit History). We handled this by:

- **Numeric columns** (Income, Loan Amount, etc.) — filled missing values with the **median** (the middle value), which is a safe, standard approach that isn't skewed by extreme outliers.
- **Text/category columns** (Gender, Married, etc.) — filled missing values with the **most common** value in that column.
- Converted "3+" dependents into a plain number (3), so it can be treated numerically.

Step 3 — Encode Text as Numbers

Machine learning models only understand numbers, not words. So we converted:

- Male/Female → 1/0
- Yes/No (Married, Self_Employed) → 1/0
- Graduate/Not Graduate → 1/0
- Property_Area (3 categories) → converted into separate 0/1 columns, one per category (called "one-hot encoding")

Step 4 — Split Into Training and Test Sets

We split our 491 applications into:

- **392 applications (80%)** — used to train the model
- **99 applications (20%)** — held back, never shown to the model during training, used only afterward to test it fairly

This split is critical — it's how we can honestly claim our accuracy numbers, since the model is being tested on applicants it has genuinely never encountered.

Step 5 — Train the Random Forest

We fed the 392 training applications (with their known Approved/Rejected answers) to scikit-learn's RandomForestClassifier, which built 200 decision trees and learned the patterns, as explained in Section 3.

Step 6 — Evaluate on the Test Set

We asked the trained model to predict outcomes for the 99 test applicants (without telling it the real answers), then compared its guesses to the actual outcomes to calculate accuracy and other scores (see Section 6).

Step 7 — Generate Charts & Save the Model

We produced 3 charts (confusion matrix, feature importance, ROC curve — explained in Section 7) and saved the trained model to a file (`loan_rf_model.joblib`) so it can be reused later without retraining.

Step 8 — Live Demo Prediction

Finally, the script picks one sample applicant from the test set and shows a real, live prediction with a confidence percentage — this is what you'll show on screen during your demo.

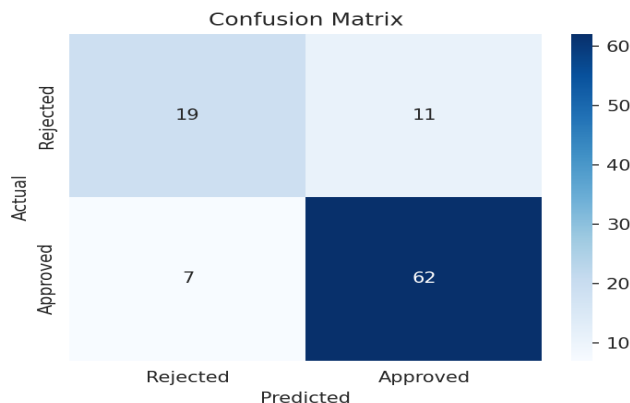
6. Our Results — What the Numbers Mean

Metric	Score	Plain-English Meaning
Accuracy	81.8%	Out of 100 applicants, the model predicts about 82 correctly
Precision	84.9%	When the model says "Approved", it's correct about 85% of the time
Recall	89.9%	Of all applicants who truly should be approved, the model correctly identifies about 90% of them
F1 Score	87.3%	A balanced average of Precision and Recall
ROC-AUC	83.2%	How well the model separates good vs risky applicants (100% = perfect, 50% = random guessing)

Is 81.8% good? Yes — for this well-studied, classic dataset, published projects typically report accuracy in the 78–86% range. We're solidly within that normal, respectable range for a first version of the model.

7. Understanding Our Charts

7.1 Confusion Matrix

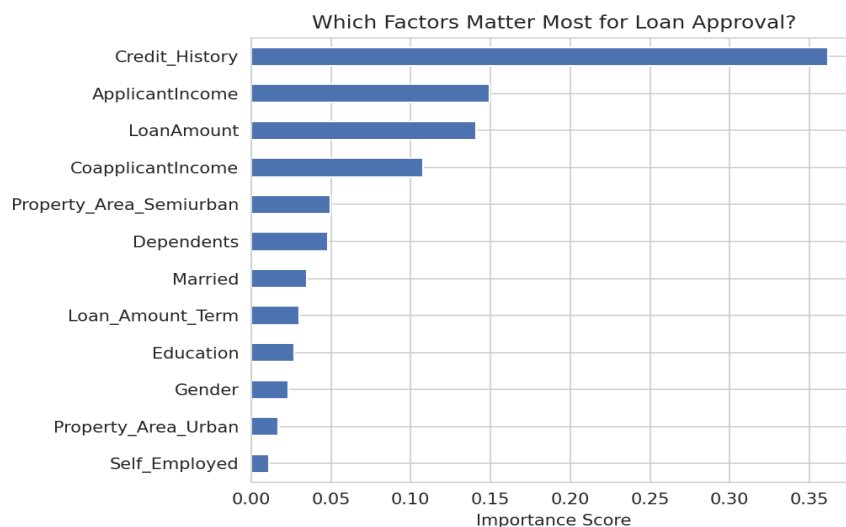


This grid shows exactly how many predictions were correct vs wrong, broken down by category:

- **Top-left & bottom-right** (diagonal) = correct predictions
- **Top-right & bottom-left** (off-diagonal) = mistakes

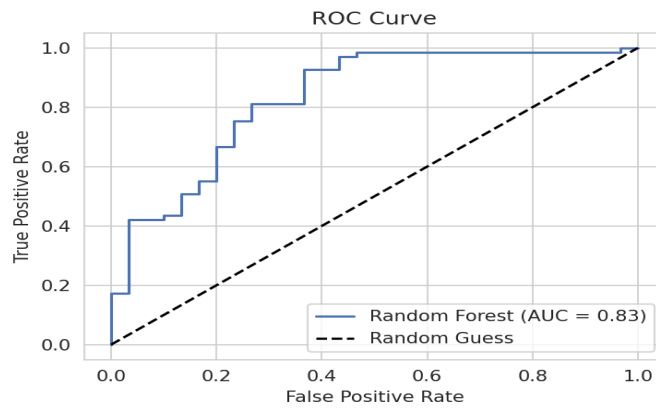
It lets us see not just "how often were we right" but specifically *which kind* of mistake the model makes more — e.g. does it wrongly reject good applicants, or wrongly approve risky ones? This matters a lot in banking, since the two types of mistakes have very different costs.

7.2 Feature Importance



This chart ranks which pieces of information the model relied on most to make its decisions. **Credit History** is by far the most influential factor — more than income or loan amount. This is a great validation point: it matches how real banks actually think, which suggests our model learned genuine, sensible patterns rather than random noise.

7.3 ROC Curve



This curve shows how well the model can tell approved and rejected applicants apart, at every possible confidence threshold. The closer the blue curve hugs the top-left corner (away from the diagonal dashed "random guess" line), the better the model is. The area underneath this curve (AUC) is our 83.2% ROC-AUC score.

8. Anticipated Questions & How to Answer Them

Q: Did you build the algorithm yourself?

A: No — Random Forest is a well-established, published algorithm. We used Python's scikit-learn library, which already implements it. What we built is a brand-new *trained model* — our own instance of it, trained on our own dataset from scratch.

Q: Why Random Forest and not another algorithm?

A: It handles a mix of numeric and categorical data well, resists overfitting better than a single decision tree, and gives us feature importance for free — useful for explaining *why* a loan was rejected, which matters for transparency in finance.

Q: How do you know the model isn't just memorizing the data?

A: We tested it on 99 applications it never saw during training (the test set), and it still got about 82% right. If it had just memorized, it would perform poorly on unseen data.

Q: What does 'class_weight=balanced' do?

A: Our dataset naturally has more approvals than rejections. This setting tells the model to pay equal attention to both outcomes instead of becoming biased toward always predicting "approved".

Q: What's next for the final submission?

A: Hyperparameter tuning (GridSearchCV) to push accuracy higher; comparing against other algorithms (Logistic Regression, XGBoost) to justify our choice; building a simple web form (e.g. with Streamlit) so a user can type in applicant details and get a live prediction; and outputting a graded "risk score" (probability) instead of a flat approve/reject.

End of guide — good luck with your presentation!